

Chapter Nine

Test-Driven Development

Testing

- **Test-Driven Development**
 - Enforces testing as part of the development process
 - A style of software development, not just a matter of testing your code
 - Tests written before the code
 - Unit test is much easier for new code than it is for existing code

Test Driven Development (TDD)

- TDD is software development approach in which test cases are developed to specify and validate what the code will do.
- In simple terms, test cases for each functionality are created and tested first and if the test fails then the new code is written in order to pass the test and making code simple and bug-free.
- Test-Driven Development starts with designing and developing tests for every small functionality of an application.
- TDD framework instructs developers to write new code only if an automated test has failed. This avoids duplication of code.

TDD Overview (1 of 3)

- Made popular by Extreme Programming
- Method of developing software not just testing software
- Software is Developed in short iterations
- Unit Tests are developed first before the code

TDD Overview (2 of 3)

- How It Works –

1. Add a Test

- Use Cases / User Stories are used to understand the requirement clearly

2. Run all tests and see the new one fail

- Ensures test harness is working correctly
- Ensures that test does not mistakenly pass

3. Write some code

- Only code that is designed to pass the test
- No additional functionality should be included because it will be untested

TDD Overview (3 of 3)

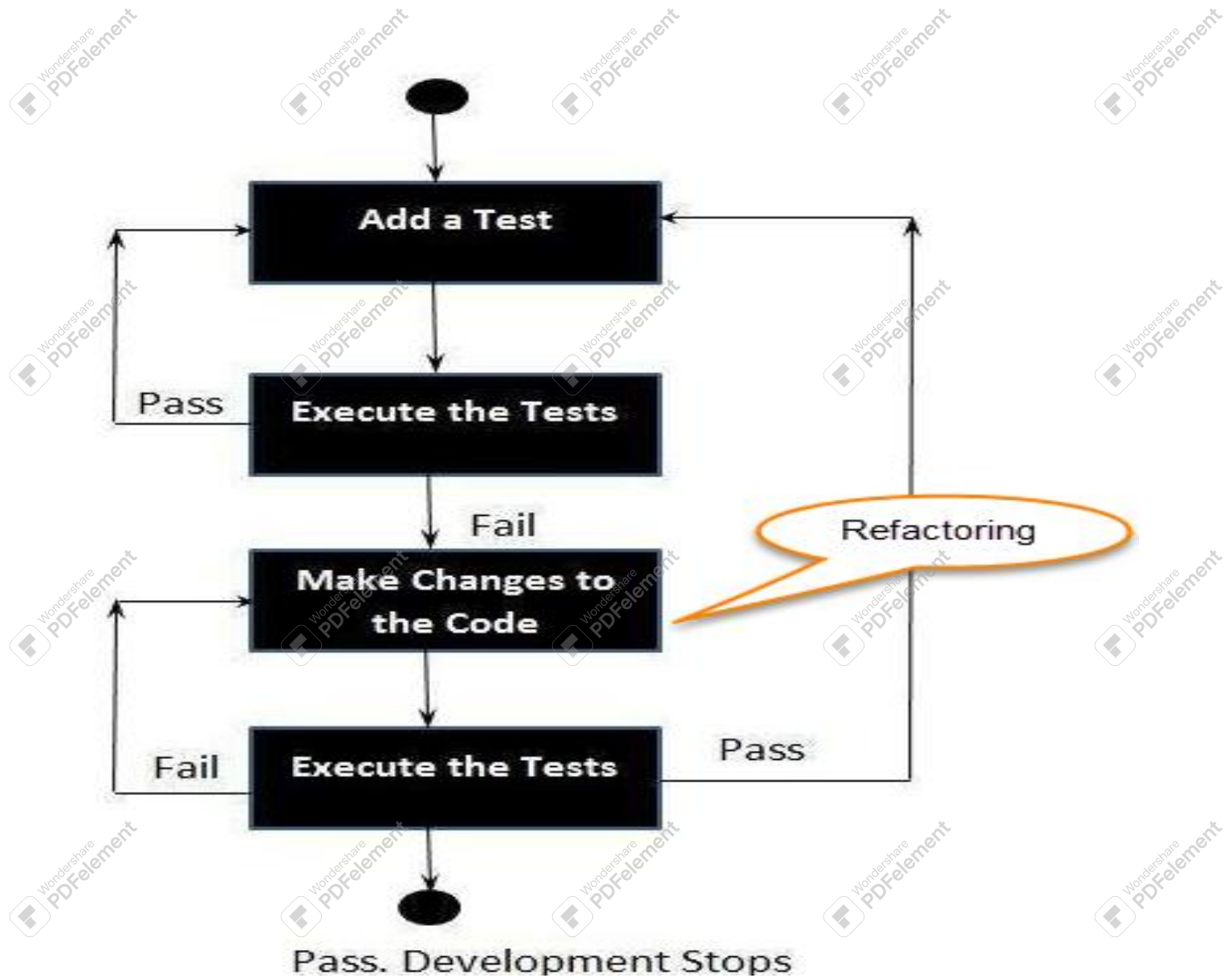
4. Run the automated tests and see them succeed

- If tests pass, programmer can be confident
- code meets all tested requirements

5. Refactor code

- Cleanup the code
- Rerun tests to ensure cleanup did not break anything

Repeat



Test Writing First

- Idea is to write tests, where each test adds some degree of functionality
- Passing the tests should indicate working code (to a point)
- The tests will ensure that future changes don't cause problems

Running Tests

- Use a test harness/testing framework of some sort to run the tests
 - A variety of ways to do this, including many existing frameworks that support unit tests
 - Junit(aimed to create an up-to-date foundation for developer-side testing on the JVM) is the most well-known, but there is similar functionality across a wide range of languages

Test framework

- Specify a test *feature*
 - Basically builds a state that can be tested
 - Set up before tests, removed afterward
- Test suite run against each feature
 - Set of tests (order should not matter) to verify various aspects of functionality
 - Described as series of assertions
- Runs all tests automatically
 - Either passes all, or reports failures
 - Better frameworks give values that caused failure

Successful Tests

- Tests should eventually pass
- You need to check that **all** tests for that unit have passed, not just the most recent.

Checklist: Test Cases

- Does each **requirement** that applies to the class or routine **have its own test case**?
- Does each element from the **design** that applies to the class or routine have its own test case?
- Has each **line of code** been tested with at least one test case?
- Have all defined-used **data-flow paths** been tested with at least one test case?
- Has the code been checked for data-flow patterns that are unlikely to be correct?

Test cases contd...

- Has a list of **common errors** been used to write test cases to detect errors that have occurred frequently in the past?
- Have all **simple boundaries** been tested: maximum, minimum, off-by-one?
- Have **compound boundaries** been tested: combinations of input data that might result in a computed variable that is too small or too large?
- Do test cases check for the **wrong kind of data**?
- Are the minimum and maximum normal **configurations** tested?
- Is **compatibility** with old data tested?
- Do test cases make hand-checks easy?

Refactoring

- As code is built, added on to, it becomes messier
- Need to go back and **rewrite/reorganize** sections of the code to make it cleaner
- Do this on a regular basis, or when things seem like they could use it
- **Only refactor after all tests are passing**
- It improves internal structure of the code, not affecting the external behavior.

Reasons to Refactor

- Code is duplicated
- A routine is too long
- A loop is too long, or too deeply nested
- A class has poor cohesion
- A class interface does not provide a consistent level of abstraction
- A parameter list has too many parameters
- Changes require parallel modifications to multiple classes
- Inheritance hierarchies have to be modified in parallel
- Case statements have to be modified in parallel

Reasons Not to Refactor

- None?
- Don't use refactoring as a cover for code and fix.
 - Refactoring is changes in **working** code that do not affect behavior.
- Avoid refactoring instead of rewriting.
 - Sometimes code just needs to be redesigned and re-implemented.

Refactoring Common Operations

- Extract Class
- Extract Interface
- Extract Method
- Replace types with subclasses
- Replace conditional with polymorphic objects
- Form template
- Introduce “explaining” variable
- Replace constructor with “factory” method
- Replace inheritance with delegation
- Replace magic number with symbolic constant
- Replace nested conditional with guard clause

When to Refactor

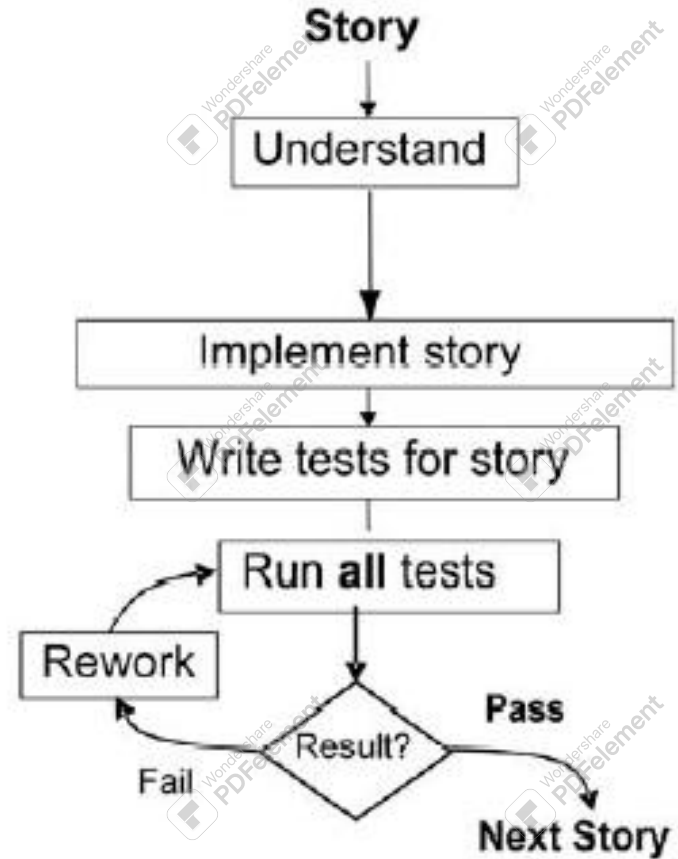
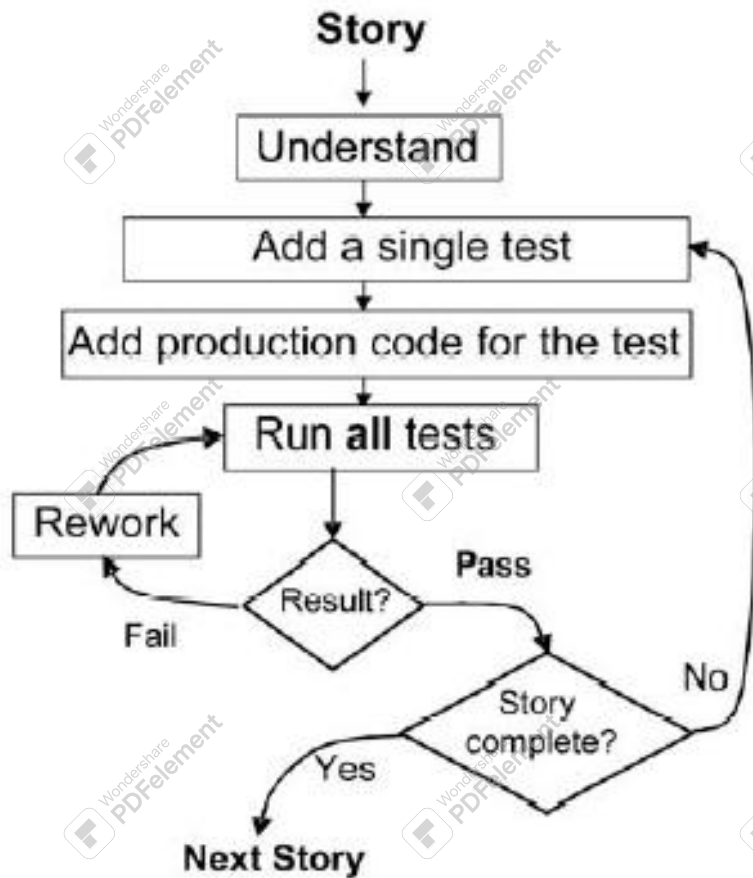
- Consider refactoring after you
 - Add a routine
 - Add a class
 - Fix a defect
 - Touch anything in the code
- Target error-prone and high complexity modules.

Test First vs. Test Last

- Pick a piece of functionality
- Write a test that expresses a small task that fails
- Write production code until test passes
- Run all tests
- Rework code until all tests pass
- Repeat

- Pick a piece of functionality
- Write production code that implements entire functionality
- Write tests to validate all functionality
- Run all tests
- Rework code until all tests pass

Test First vs. Test Last



Research Study #1

- “On the Effectiveness of Test-first Approach to Programming”
 - H. Erdogmus, T. Morisio – 2005 [1]

Research Study #1 Background

- Subjects are 3rd year students
 - Had 8 week intensive Java course
 - Learned unit testing with JUnit
 - Learned basic design concepts
 - Learned object orientation
- Two groups –
 - Control Group – Test-Last
 - TDD Group – Test-First
- Each group developed a bowling game

Research Study #1 - Hypotheses

- Test-First programmers write more test per unit of programming effort [1]
- Test-First programmers produce high quality programs [1]
- Test-First programmers are more productive overall [1]
- Writing more tests improve quality [1]
- Writing more tests increase productivity [1]

Research Study #1 Results

- Quality
 - Test-First subjects did not result in an increase in quality
- Productivity
 - Better task understanding
 - Better task focus
 - Faster learning
 - Lower rework effort
 - Wrote more test per unit of programming
 - Higher number of programmer tests lead to proportionally higher levels of productivity

Research Study #2

- “A structured experiment of test-driven of test-driven development”
 - B. George, L. Williams – 2003 [2]

Research Study #2 Background

- Programmers used pair-programming practices
- Two groups –
 - Control Group used design-develop-test (waterfall) approach
 - TDD Group
- Each group developed a bowling game

Research Study #2 Hypotheses

- TDD practices would yield code that's superior to code developed with waterfall-like practices [2]
- TDD developers would develop code faster than developers using waterfall-like practices [2]

Research Study #2 – Results

- **Quality**
 - TDD pairs' code passed approximately 18% more test cases than the control groups [2]
 - TDD practices appear to yield code with superior external quality [2]
- **Productivity**
 - TDD programmers took approximately 16% more time than the control group programmers [2]
- **Code Coverage**
 - TDD programmers test cases achieved a mean of 98% method, 92% statement and 97% branch coverage [2]

Research #3

- “Assessing Test-Driven Development at IBM”
 - E. Maximilien, L. Williams - 2002

Research Study #3 Background

- IBM Retail Store Solutions (RSS) is a founding member of Java for Point of Sale (JavaPOS) specification [3]
- The JavaPOS defect rate was not being reduced with each revision of the deliverable [3]
- The unit test process was not disciplined and was done as an afterthought [3]

Research Study #3

- What did IBM want to measure with TDD? [3]
 - Defect Rate
 - Productivity
 - Test Frequency
 - Design
 - Integration
- 80% of the important classes were covered by automated unit testing [3]

Research Study #3 Results (1 of 2)

- Defect Rate
 - Approximately a 50% reduction in defect density
 - 7.0 errors/KLOC (thousands of lines of code) before TDD
 - 3.7 errors/KLOC after TDD
- Productivity
 - With TDD the productivity was below the 400 LOC/person-month estimate
- Test Frequency
 - 2500 Automated Tests – Run Daily
 - 400 Interactive Tests – Rarely Run

Research Study #3 Results (2 of 2)

- Design
 - TDD practice aided in producing a product that would be more easily incorporate late changes
- Integration
 - Daily Integration
 - Problems surfaced earlier

TDD Benefits (1 of 3)

- Instant Feedback
 - Developer knows instantly if new code works and if it interferes with existing code [1]
- Better Development Practices
 - Encourages the programmers to decompose the problem into manageable, formalized programming tasks
 - Provides context in which low-level design decisions are made
 - By focusing on writing only the code necessary to pass tests, designs can be cleaner and clearer than is often achieved by other methods

TDD Benefits (2 of 3)

- Quality Assurance
 - Having up-to-date tests in place ensures a certain level of quality
 - Enables continuous regression testing
 - TDD practices drive programmers to write code that is automatically testable
 - Whenever a software defect is found, unit test cases are added to the test suite prior to fixing the code

TDD Benefits (3 of 3)

- Lower Rework Effort
 - Since the scope of a single test is limited, when the test fails, rework is easier
 - Eliminating defects early in the process usually avoids lengthy and tedious debugging later in the project
 - “Cost of Change” is that the longer a defect remains the more difficult and costly to remove

TDD Limitations (1 of 2)

- Counterproductive and hard to learn [1]
- Difficult in Some Situations
 - GUIs, Relational Databases, Web Service
 - Requires mock objects
- TDD does not often include an upfront design [2]
 - Focus is on implementation and less on the logical structure

TDD Limitations (2 of 2)

- Difficult to write test cases for hard-to-test code
 - Requires a higher level of experience from programmers [2]
- TDD blurs distinct phases of software development
 - design, code and test [2]

TDD Survey

Results of a survey conducted by 24 professional programmers [2]

Concern/Sub-concerns	% Agree
Productivity-Aggregate	78
Facilitates better requirements	88
Reduces debugging effort	96
Reduces development time	50
Effectiveness-aggregate	80
Yields higher code quality	92
Promotes simpler design	79
Is noticeably effective	71
Difficulties in adoption – aggregate	40
Getting into TDD mindset	56
Lack of upfront design a hindrance	23

Summary

- The research studies results are inconsistent
 - Quality
 - Productivity
- TDD can be effective if you consider
 - Goals of your software group
 - Kind of software being developed
 - The skill level and experience of your developers

Thank You!!

